CSEN 296B-2 | Week 9 Lab Submission

AstraNotes Test Improvement Log

Student Name: Atishay Jain | **Date:** May 30, 2026 | **Project:** AstraNotes | **Technical Path:** Python
Repository: github.com/Atishay8192261/astranotes (private) | **Pull Request:** #2 (merged) | **Suite after this lab:** 70 passed, 5 skipped (legacy CLI quarantined)

Feature / requirement reviewed

SPR-01 (private note bodies encrypted with Fernet before any disk write) as enforced by *test_create_private_note_encrypts_body_on_disk* in *tests/test_note_manager.py*. This is the single load-bearing test for the project's most security-critical requirement: a note marked *is_private=True* never lands plaintext on disk. The test has existed since Sprint Zero and has shipped through every milestone.

I picked it because SPR-01 is the requirement my grade and my user's privacy both ride on. If any test in the suite deserves to be strong, it is this one.

The weakness — a brittle assertion masquerading as a security check

The original test was five lines:

```
def test_create_private_note_encrypts_body_on_disk(manager, tmp_path):
    manager.create_note(title="secret", body="meet at 5", is_private=True)
    raw = list(tmp_path.glob("*.json"))[0].read_text()
    assert "meet at 5" not in raw
    assert '"body_encoding": "base64"' in raw
```

Two assertions, both phrased as questions about the *implementation* rather than the *property*:

• **(1)** *assert "meet at 5" not in raw* is the right idea, but pinning a single short phrase gives false confidence. *"5"* could appear in an ISO timestamp; *"meet"* could be a tag. The test would also miss a regression that leaks a *different* substring of the plaintext while suppressing the exact phrase. The assertion fits the literal SPR-01 sentence but does not probe its spirit.

• **(2)** *assert '"body_encoding": "base64"' in raw* is the bigger problem: it pins a *private serialization detail*. If we evolved the storage format - added a *"v": 2* envelope, switched encoding to hex, wrapped the ciphertext in a per-note salt - this assertion would break **even though SPR-01 still held**. Conversely, if a regression switched private bodies to a plaintext-with-*body_encoding=base64*-wrapper (base64-encoded the cleartext), the assertion would still pass. It is testing the *shape* of the right answer, not the *answer*.

This is the canonical 'misleading coverage confidence' failure mode from the rubric.

The improvement

Rewritten to assert the property directly:

```
def test_create_private_note_encrypts_body_on_disk(manager, tmp_path):
    """Gap #4 fix: assert the SPR-01 property, not the format."""
    import json
    plaintext = "meet rendezvous coordinates xyz123"
    manager.create_note(title="secret", body=plaintext, is_private=True)
    payload = json.loads(list(tmp_path.glob("*.json"))[0].read_text())

    # 1) Plaintext never appears in the on-disk body field, AND no
    #    token of the plaintext appears either - so a leak of any
    #    substring longer than a single word is caught.
    assert plaintext not in payload["body"]
    for word in plaintext.split():
        assert word not in payload["body"], (
            f"plaintext token '{word}' leaked to disk")

    # 2) The body still decrypts back to the original under the
    #    same key. If a regression "encrypts" by base64-ing
    #    cleartext, step 1 catches it; if it scrambles into
    #    something undecryptable, step 2 catches it.
    notes = manager.list_notes()
    assert [n.body for n in notes] == [plaintext]
```

Three things change. **First**, the assertion targets the JSON *body* field specifically (*payload["body"]*) instead of the whole file text - this is what lets the per-word check work without false positives from *created_at* containing the substring *"at"* (a real failure I hit while writing the test, see AI section below). **Second**, the per-word loop makes the leak detection robust to any substring leak longer than a single token. **Third**, the test asserts the *round-trip* - that the ciphertext decrypts back to the original - which is the only assertion that catches a regression that encodes cleartext rather than encrypting it.

The new test is decoupled from the storage envelope format: changing *body_encoding* from *base64* to *hex*, or wrapping the payload in a versioned envelope, would not break this test. A real SPR-01 regression would.

Mocking — helping or hiding risk?

This codebase's discipline around mocking is one of the things I deliberately did not change. The fixture for this test uses a **real** *JsonFileRepository*, a **real** *PrivacyService* (with *PrivacyService.generate_key()* for the test key), and a *tmp_path* from pytest. There are no mocks anywhere in the privacy or persistence layers, by convention since Week 7.2.

The pattern is right. SPR-01 is a property about what hits the disk; a test of it that mocks the disk is testing the wrong thing. The one place I had to be careful was the temptation - when I added a 'decryption fails during toggle' test in this same PR - to monkeypatch *PrivacyService.decrypt* to raise. I resisted that and used two distinct *PrivacyService* instances with different keys, mirroring the existing pattern. A *decrypt* that raises because the wrong key was used is a different code path than a *decrypt* that raises because I told it to. The real-key approach exercises Fernet's actual *InvalidToken* flow.

The single place mocking *did* help this PR was in the new repository tests for filesystem failures. There I monkeypatched *pathlib.Path.write_text* to raise *OSError("disk full")*. This is mocking at the **boundary** where the real failure surfaces a real branch in the code under test (the *except OSError:* clause that translates to *PersistenceError*). That branch was 100% uncovered before; nothing short of mocking would have reached it without an actually-failing disk. Mocking at the **OS boundary** to exercise an internal error-translation branch is helping; mocking the **service under test** to make it raise on cue would have been hiding.

A meaningful coverage gap that still matters

The improved test proves that the *first* write of a private note never leaks plaintext, and that a fresh *list_notes()* round-trips. What it does **not** prove is that the *whole lifecycle* of a private note preserves the property - specifically, that an **edit** to a private note also re-encrypts on every save, that a *set_private(False)* toggle correctly *replaces* the ciphertext with plaintext (not appends), and that a second update doesn't accidentally double-encrypt the body.

I added partial coverage for the edit and toggle paths in the same PR (*test_update_preserves_encryption_for_private_note*, *test_set_private_true_encrypts_body_on_disk*, *test_set_private_false_decrypts_and_writes_plaintext*), but none of those check the *file content after multiple edits*. A user who creates a private note, edits it once, then edits it again is exercising a path the suite still asserts only at the in-memory API level. The next test I would add - and the reason I am calling this out instead of slipping it in silently - is *test_repeated_edits_keep_private_body_encrypted_on_disk* that creates, updates, updates again, and re-reads the raw JSON each time to confirm the body field is never plaintext.

The coverage gap that does *not* matter and that I am deliberately not chasing: testing for plaintext leakage in OS-level metadata (swap files, journaled filesystems, FUSE caches). That is an OS-layer threat model that ADR-005's threat model explicitly excludes, and writing tests for it would be theater.

How AI helped, and what I accepted / changed / rejected

Accepted. Claude proposed the per-word loop (*for word in plaintext.split()*) as the way to broaden the leak check beyond a single-phrase substring. That is the structural improvement that makes the new test stronger than the old one, and I would not have written it on the first try.

Changed. Claude's first draft tried to assert against the entire raw file text - exactly the trap I was trying to escape. The test failed when the plaintext token *"at"* matched the substring *"created_at"* in the timestamp field. The fix - *json.loads* the payload and assert against *payload["body"]* instead of the whole string - is mine; the failure made the bug obvious in a way reading the code didn't. The AI's draft had the right idea but the wrong target.

Rejected. Claude also suggested adding a *fuzz* test that generated 100 random plaintexts and asserted the property holds across all of them. I declined. The property is structural - Fernet either encrypts or it doesn't - so 100 random inputs prove nothing that 1 well-chosen input doesn't, and the slower test would just add CI time. The same suggestion would be appropriate for a *parsing* function where input variety matters; here it is overkill for show, not for signal.

On framing the rubric pieces. I drafted the 'mocking helping vs hiding' paragraph myself, then asked Claude to push back on it. It correctly pointed out that I had not justified *why* mocking *pathlib.Path.write_text* was OK while mocking *PrivacyService.decrypt* would not be. I tightened the distinction - boundary-mocking the OS to exercise an internal error branch vs. service-mocking the code under test to fake an outcome - and that is the version that ended up in the doc. The framing decision is mine; the request to make it explicit is what AI added.

The honest limit. AI is good at proposing the structural move and at catching the lazy phrasing. It is not good at deciding what to test, what coverage is theater, or which gap is actually load-bearing for a given threat model. Those calls - the ones the rubric is actually asking about - have to be mine.